

Rapport de projet

Programmation orientée objet en Python

**Daniil Maksymenko, Mohamed Yacine Chabane, Pola
Szopka**
M1 MEDH



Sorbonne Université
4 Place Jussieu, 75005 Paris
November 16, 2025

Structure Globale

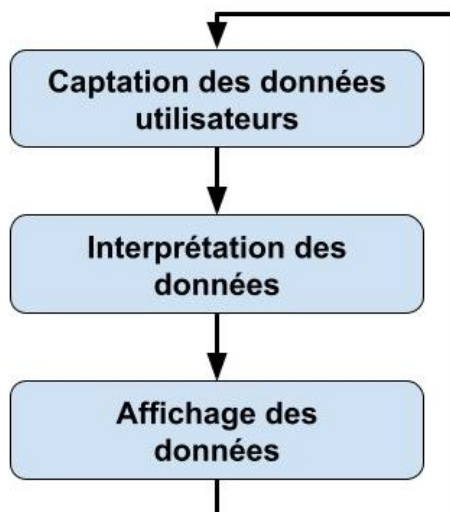
Pour la structure globale, nous avons choisi d'avoir deux classes principales : la classe GUI (Graphical User Interface) et la classe Engine.

La classe GUI est la seule à manipuler la bibliothèque Python Pygame, et s'occupe de la gestion des entrées utilisateurs (les touches tapées au clavier), et de l'affichage du jeu.

La classe Engine s'occupe de gérer toute la logique du jeu, en interprétant les données utilisateur pour décider de ce qui va être affiché à l'écran.

C'est

Voici un schéma très général de notre algorithme :



Ces deux blocs principaux communiquent entre eux à l'aide d'un dictionnaire appelé "input". Il contient, entre autres, la position du joueur, une liste des salles déjà tiré, la position du curseur si le joueur a un choix à faire, etc.

Nous avons choisi cette méthode, car elle nous a permis de simplifier les échanges entre les deux blocs et d'éviter un grand nombre d'attributs de classe dans les classes Engine et GUI.

Classes Principales et Relations

Les chambres

Pour gérer les différentes chambres et leurs effets, nous avons fait de Rooms une classe abstraite. Elle force toutes les classes-filles à implémenter la méthode `apply_effects()`. Cette méthode est ensuite utilisée dans Engine, à chaque fois que l'on entre dans une pièce. Cela assure le polymorphisme : chaque pièce applique ses effets uniques (ex. plus de pas pour une pièce violette) sans devoir modifier le code central.

Chaque type de pièces (ex les pièces rouges) hérite de la classe Rooms. Ensuite chaque type de pièces spécifique (ex : la salle de bains) hérite de son type de pièce.

Pour plus de lisibilité, nous avons choisi de mettre chaque type de chambres dans son propre fichier python. Enfin toutes les instances de chambress sont rassemblés dans le fichier Pioche.py.

L'inventaire

Pour gérer notre inventaire, nous avons notre classe inventaire, pour pouvoir gérer le nombre de pas, de pièces de gemmes, etc.

Pour les objets permanents, nous avons choisi de créer une classe Object, qui gères les objets permanents ou consommables, comme le marteau ou les pommes.

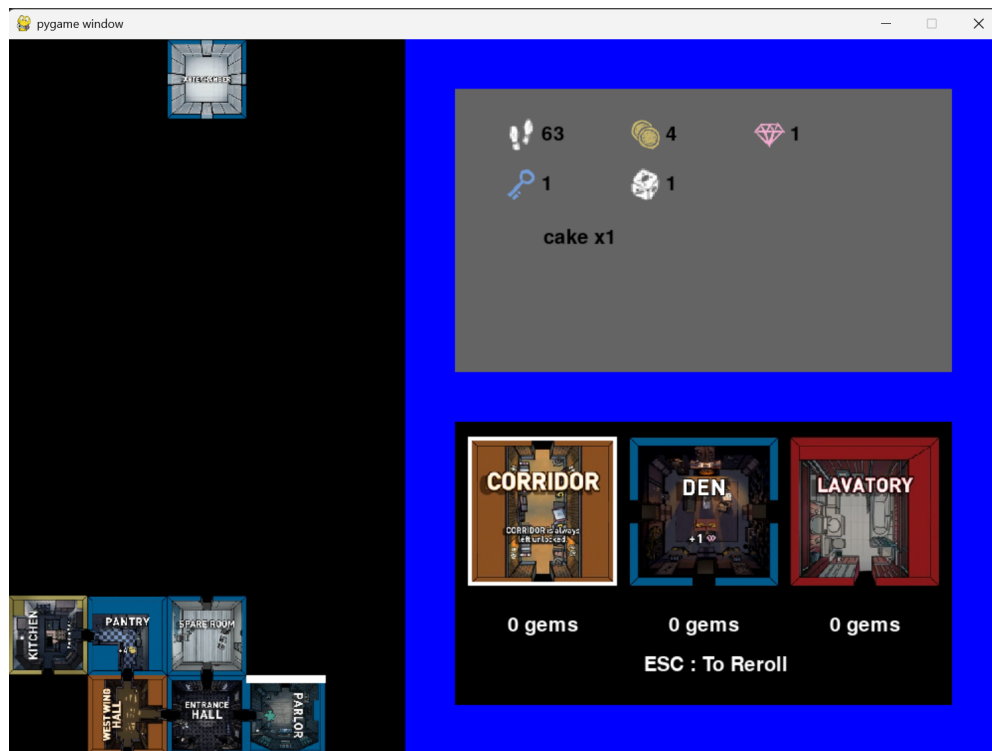
Nous avons aussi la classe Sellables, qui répertorie les objets 'vendable' dans le magasin, et qui les place dans l'inventaire une fois acheter

Gestion de l'Aléatoire et des Probabilités

Pour le tirage aléatoire des salles, nous avons utilisé la fonction de numpy : `numpy.random.choice()`, qui nous a permis de piocher 3 salles sans remise, avec une probabilité de tirage des salles en fonction de leur rareté.

Une autre fonctionnalité qui relève de l'aléatoire est le verrouillage des portes. En effet, lors de la création d'une salle, il y a une certaine probabilité que celle-ci soit verrouillée, à simple ou à double tour. Cette probabilité augmente en fonction du nombre de salles : elle est de $2 * \text{nb de chambres} \%$ pour un verrouillage à simple tour et de $\text{nb de chambres} \%$ pour un verrouillage à double tour.

Interface utilisateur



Voici un exemple de l'interface utilisateur en milieu d'une partie.

On peut voir sur la partie de gauche, le plateau de jeu, avec les différentes salles qui ont été tirées. La position du joueur est indiquée par le rectangle blanc.

Sur la partie droite, en haut, on peut voir l'état actuel de l'inventaire de l'utilisateur, avec les objets ordinaires du jeu, comme les pièces ou les pas, et des objets que l'on gagne dans les pièces comme ici la pomme.

Enfin, en bas à droite, on retrouve la partie choix. En effet, c'est ici que vont s'afficher les choix possibles pour l'utilisateur. Il va s'agir du choix des salles, du choix de déverrouillage d'une porte, ou du choix dans les magasins.

Pour pouvoir interagir avec le jeu, l'utilisateur peut utiliser les touches suivantes :

- Flèches haut, bas, droite, gauche pour se déplacer
- Return ou Espace pour accepter un choix
- Echap pour revenir en arrière
- P/M/O/L/K pour consommer la nourriture,
- E/C/V pour interagir avec les objets (pelle/marteau/clé sur coffres/trous)

Des Compromis

Par manque de temps, nous n'avons pas implémenté les fonctionnalités suivantes :

- Les salles qui dispersent les objets dans d'autres salles
- Les pièces qui ajoutent d'autres pièces à la pioche